

汎用組織向け会員アプリ仕様書

1. アプリ概要

本アプリは、特定の体操支部専用ではなく、さまざまな組織で利用できる汎用型の会員アプリである。

従来の開発では機能が多岐にわたり構成が複雑になったため、本アプリでは機能を厳選し、以下の4機能に特化したシンプルで運用しやすい構成とする。

- ・会員登録
- ・顔認証ログイン
- ・メッセージ配信
- ・ビデオコンテンツ販売

本アプリは、組織が会員に対して情報を提供し、限定機能を安全に公開し、動画コンテンツを販売・提供することを目的とする。

2. 開発目的

2-1. 目的

- ・組織ごとの会員管理をアプリで簡単に行えるようにする
- ・会員だけが閲覧できる専用ページを安全に提供する
- ・管理者から会員へメッセージを配信できるようにする
- ・有料動画コンテンツを販売し、購入者のみ視聴できる仕組みを実現する

2-2. コンセプト

- ・シンプル
- ・安全
- ・拡張しやすい
- ・特定業種に依存しない

2-3. 想定利用組織

- ・健康教室
 - ・サークル
 - ・協会
 - ・学習団体
 - ・会員制サービス事業者
 - ・地域コミュニティ
-

3. 対象ユーザー

3-1. 一般利用者

- ・アプリをダウンロードして利用する会員

- まだ会員登録していない一般利用者

3-2. 管理者

- 会員申請を確認する担当者
 - メッセージを作成・送信する担当者
 - ビデオコンテンツを登録・公開管理する担当者
-

4. アプリ構成

本システムは、以下の2つで構成する。

4-1. 会員アプリ

会員または一般利用者が利用するアプリ。

4-2. 管理機能

管理者が会員情報、メッセージ、動画商品を管理するための管理画面または管理アプリ。

※ 初期段階では、会員アプリ側の実装を優先し、管理機能は必要最小限から構築する。

5. 機能一覧

5-1. 会員アプリ機能

1. 会員登録機能
2. 顔認証ログイン機能
3. メッセージ閲覧機能
4. ビデオ一覧表示機能
5. 動画購入・購入状態確認機能
6. 購入済み動画の視聴機能
7. 会員情報表示機能
8. ログアウト機能

5-2. 管理機能

1. 会員登録申請確認
 2. 会員承認・管理
 3. メッセージ送信
 4. 動画商品登録
 5. 動画公開設定
 6. 購入者管理
-

6. 会員アプリ画面仕様

6-1. 起動画面

概要

アプリ起動時に表示される画面。

表示内容

- 組織名
- ロゴ画像またはメイン画像
- 読み込み中表示

動作

- 組織設定を読み込む
 - 未ログイン時はトップ画面へ遷移
 - ログイン済みかつ認証条件を満たす場合は会員ページへ遷移可能
-

6-2. トップ画面

概要

アプリ利用開始時の入口となる画面。

表示内容

- 組織紹介
- お知らせ用の簡易案内文
- 会員登録ボタン
- ログインボタン
- ビデオ案内導線

動作

- 未登録者は会員登録へ進む
 - 登録済みユーザーは顔認証付きログインへ進む
-

6-3. 会員登録画面

目的

新規利用者が会員登録を行う。

入力項目案

- 氏名

- ・フリガナ
- ・生年月日
- ・メールアドレス
- ・電話番号
- ・任意の会員番号
- ・所属情報または区分
- ・備考

動作

- ・入力内容を送信する
- ・登録申請データを保存する
- ・申請完了メッセージを表示する
- ・承認後に会員として利用可能になる

補足

- ・初期版では申請制にする
- ・自動承認は行わない

6-4. 顔認証ログイン画面

目的

登録済み会員が安全に会員ページへ入る。

認証方式

- ・Face ID または Touch ID
- ・端末が生体認証非対応の場合は端末パスコード認証

動作

- ・初回ログイン時に認証設定を有効化
- ・以後、会員ページに入る際に認証を要求
- ・認証成功後に会員専用ページを表示

制限

- ・未承認会員は会員ページへ進めない
- ・未登録ユーザーは認証のみで会員ページへ入れない

6-5. 会員ページ

概要

ログイン後のメイン画面。

表示内容

- 会員名
- 会員ステータス
- メッセージボタン
- ビデオコンテンツボタン
- ログアウトボタン

補足

- 必要最低限のメニュー構成とする
 - 将来拡張しやすい設計にする
-

6-6. メッセージ一覧画面

目的

管理者から配信されたメッセージを会員が閲覧する。

表示内容

- 件名
- 配信日時
- 未読 / 既読表示

動作

- 一覧を新しい順に表示
 - タップで詳細画面へ遷移
 - 詳細を開いた時点で既読に変更する
-

6-7. メッセージ詳細画面

表示内容

- 件名
- 本文
- 配信日時
- 添付情報（将来拡張）

補足

- 初期版ではテキスト中心で構成する
 - 画像やPDF添付は将来拡張機能とする
-

6-8. ビデオ一覧画面

目的

販売中または購入済みの動画コンテンツを一覧表示する。

表示内容

- サムネイル
- タイトル
- 説明文
- 価格
- 購入状態

動作

- 公開中動画のみ表示
 - 購入済みか未購入かを判定して表示を切り替える
-

6-9. ビデオ詳細画面

表示内容

- タイトル
- 説明文
- サムネイル
- 価格
- 購入ボタンまたは視聴ボタン

動作

- 未購入の場合は購入導線を表示
 - 購入済みの場合は視聴ボタンを表示
-

6-10. 動画視聴画面

目的

購入済み会員のみが動画を再生できるようにする。

動作

- 動画プレイヤーを表示
- 再生URLは安全に管理する
- 未購入ユーザーは視聴不可

補足

- 動画配信基盤は Vimeo 等の外部サービス利用を前提とする
 - アプリ内表示は WebView または安全な埋め込み再生方式を採用する
-

7. 管理機能仕様

7-1. 会員申請管理

目的

新規登録申請を確認し、承認・保留管理を行う。

必要機能

- 申請一覧表示
- 申請詳細表示
- 承認
- 却下または保留

管理項目

- 氏名
 - 申請日時
 - 連絡先
 - 承認状態
-

7-2. 会員管理

必要機能

- 会員一覧表示
 - 会員検索
 - 有効 / 無効状態変更
 - 購入状況確認
-

7-3. メッセージ配信管理

必要機能

- メッセージ作成
- 会員向け配信
- 一斉送信
- 配信履歴確認

メッセージ項目

- 件名
 - 本文
 - 配信日時
 - 公開状態
-

7-4. ビデオコンテンツ管理

必要機能

- 動画商品登録
 - タイトル編集
 - 説明編集
 - 価格設定
 - 公開 / 非公開切替
 - 動画URL管理
 - サムネイル登録
-

7-5. 購入管理

必要機能

- 誰がどの動画を購入したか確認
 - 購入日時確認
 - 購入状態の手動調整
-

8. 権限仕様

8-1. 一般利用者

- トップ画面閲覧
- 会員登録申請
- 公開範囲内の案内閲覧

8-2. 承認済み会員

- 顔認証ログイン
- 会員ページ利用
- メッセージ閲覧
- 購入済み動画視聴

8-3. 管理者

- 会員管理
- メッセージ送信

- 動画商品管理
 - 購入管理
-

9. データ設計概要

本アプリでは、主に以下のデータを管理する。

9-1. organizations

組織情報を管理する。

想定項目:

- organizationId
- name
- logoURL
- openingImageURL
- description
- isActive
- createdAt
- updatedAt

9-2. memberRegistrations

会員登録申請情報を管理する。

想定項目:

- registrationId
- organizationId
- name
- kana
- birthDate
- email
- phone
- memberCode
- category
- note
- status
- createdAt
- updatedAt

9-3. members

承認済み会員情報を管理する。

想定項目:

- memberId

- organizationId
- uid
- name
- kana
- birthDate
- email
- phone
- memberCode
- status
- biometricEnabled
- createdAt
- updatedAt

9-4. messages

会員向けメッセージを管理する。

想定項目:

- messageId
- organizationId
- title
- body
- targetType
- publishedAt
- isPublished
- createdAt
- updatedAt

9-5. messageReads

メッセージ既読情報を管理する。

想定項目:

- readId
- organizationId
- messageId
- uid
- readAt

9-6. videos

販売用動画情報を管理する。

想定項目:

- videoId
- organizationId
- title
- description

- thumbnailURL
- videoProvider
- videoSourceId
- price
- isPublished
- createdAt
- updatedAt

9-7. purchases

購入情報を管理する。

想定項目:

- purchaseId
- organizationId
- uid
- videoId
- paymentStatus
- purchasedAt
- expiresAt

10. 非機能要件

10-1. セキュリティ

- 会員専用情報は未認証ユーザーに表示しない
- 会員ページへの遷移には本人認証を必須とする
- 購入済み判定のない動画再生は禁止する
- 管理機能は管理者権限のみ利用可能とする

10-2. 操作性

- 高齢者でも操作しやすい画面設計
- ボタン数を最小限に抑える
- 1画面1目的を意識する

10-3. 拡張性

- 将来的に複数組織へ展開できる構造にする
- カテゴリ配信、添付ファイル、イベント機能などを後から追加しやすくする

11. 初期開発範囲

今回の初期開発では、以下を実装対象とする。

必須実装

- 組織情報表示
- 会員登録申請
- 管理者承認
- 顔認証ログイン
- 会員ページ
- メッセージ一覧 / 詳細
- 動画一覧 / 詳細
- 購入状態判定
- 購入済み動画視聴

初期版では簡略化する内容

- 複雑なカテゴリ分岐
- 支部分岐
- 日記
- スケジュール
- PDF添付
- 画像添付
- 個別メッセージ高度配信
- 本部管理機能

12. 将来拡張案

- 複数組織一括管理
- 組織ごとのテーマカラー変更
- 添付ファイル付きメッセージ
- イベント管理
- 会費決済
- 定期購読型動画配信
- お問い合わせ機能
- 管理者複数権限対応
- Web管理画面対応

13. 推奨開発方針

13-1. 方針

本アプリは、過去の複雑な構成を引き継がず、必要最小限の新設計で作り直す。

13-2. 理由

- 不要機能を持ち込まないため
- データ構造を整理しやすいため
- 不具合原因の切り分けがしやすいため
- 将来の汎用展開に向けて再利用しやすいため

13-3. 実装順序

1. 組織情報読み込み
2. 会員登録
3. 承認済み会員判定
4. 顔認証ログイン
5. メッセージ機能
6. 動画販売機能
7. 購入管理
8. 管理機能整備

14. まとめ

本アプリは、さまざまな組織で活用できる汎用的な会員アプリとして設計する。

初期段階では、以下の4機能に集中する。

- 会員登録
- 顔認証ログイン
- メッセージ
- ビデオコンテンツ販売

これにより、過去の複雑な仕様を整理しながら、実用性が高く、管理しやすく、将来の拡張にも耐えられるアプリ基盤を構築する。

Firestore設計図 完成版

15. 設計方針

今回の Firestore は、**特定支部向けではなく複数の組織に転用できる汎用構成**を前提とする。

ただし、初期版では複雑な多段階権限や細かな分類を入れすぎず、以下の方針で設計する。

- 最上位に `organizations` を置く
- すべての業務データは原則として `organizations/{organizationId}/...` 配下で管理する
- 会員申請、承認済み会員、メッセージ、動画、購入情報を分離する
- 会員アプリが読むデータと管理側が更新するデータを整理する
- 将来の複数組織展開に対応しやすい形にする

16. 全体コレクション構成

```
organizations/{organizationId}
├── settings/public
├── admins/{uid}
└── memberRegistrations/{registrationId}
```

```
├─ members/{uid}
├─ messages/{messageId}
├─ videos/{videoId}
├─ purchases/{purchaseId}
└─ system/version
```

補足

- `organizationId` は組織ごとの一意ID
- 初期版では、1アプリ=1組織運用でもよい
- 将来、同じ設計で複数組織へ横展開できる

17. ドキュメント配置詳細

17-1. organizations/{organizationId}

役割

組織の基本情報を保持する親ドキュメント。

想定フィールド

- `name` : 組織名
- `description` : 組織説明
- `logoURL` : ロゴ画像URL
- `openingImageURL` : 起動時画像URL
- `supportEmail` : 問い合わせ先メール
- `isActive` : 組織利用状態
- `createdAt` : 作成日時
- `updatedAt` : 更新日時

サンプル

```
{
  "name": "サンプル組織",
  "description": "会員向け情報配信アプリ",
  "logoURL": "https://...",
  "openingImageURL": "https://...",
  "supportEmail": "info@example.com",
  "isActive": true,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

17-2. organizations/{organizationId}/settings/public

役割

会員アプリ起動時に参照する公開設定。

想定フィールド

- `displayName` : 表示用組織名
- `openingEnabled` : 起動画面表示の有無
- `openingImageUrl` : 起動画面画像
- `themeColorHex` : テーマカラー
- `registrationEnabled` : 会員登録受付可否
- `videoSalesEnabled` : 動画販売機能の利用可否
- `messageEnabled` : メッセージ機能利用可否
- `updatedAt` : 更新日時

サンプル

```
{
  "displayName": "サンプル組織",
  "openingEnabled": true,
  "openingImageUrl": "https://...",
  "themeColorHex": "#2E7D32",
  "registrationEnabled": true,
  "videoSalesEnabled": true,
  "messageEnabled": true,
  "updatedAt": "Timestamp"
}
```

17-3. organizations/{organizationId}/admins/{uid}

役割

管理者権限を持つユーザーを管理する。

ドキュメントID

- Firebase Authentication の `uid`

想定フィールド

- `name` : 管理者名
- `email` : メールアドレス
- `role` : 権限種別
- `isActive` : 有効状態
- `createdAt` : 作成日時
- `updatedAt` : 更新日時

role の初期想定

- owner
- admin
- staff

サンプル

```
{
  "name": "管理者",
  "email": "admin@example.com",
  "role": "owner",
  "isActive": true,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

17-4. organizations/{organizationId}/memberRegistrations/{registrationId}

役割

一般利用者から送信された会員申請を保存する。

ドキュメントID

- 自動採番

想定フィールド

- organizationId : 所属組織ID
- name : 氏名
- kana : フリガナ
- birthDate : 生年月日
- email : メールアドレス
- phone : 電話番号
- memberCode : 任意の会員番号
- category : 所属区分または会員区分
- note : 備考
- status : 申請状態
- createdByUid : 申請時の認証UID
- reviewedByUid : 審査管理者UID
- reviewedAt : 審査日時
- createdAt : 申請日時
- updatedAt : 更新日時

status の初期想定

- pending
- approved
- rejected

サンプル

```
{
  "organizationId": "org001",
  "name": "山田太郎",
  "kana": "ヤマダタロウ",
  "birthDate": "1980-05-10",
  "email": "sample@example.com",
  "phone": "09012345678",
  "memberCode": "A-001",
  "category": "一般会員",
  "note": "紹介入会",
  "status": "pending",
  "createdByUid": "authUid123",
  "reviewedByUid": null,
  "reviewedAt": null,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

運用ルール

- 会員アプリは新規作成のみ可能
- 管理者は status を更新可能
- 承認時に members/{uid} を作成する

17-5. organizations/{organizationId}/members/{uid}

役割

承認済み会員の本データ。

ドキュメントID

- Firebase Authentication の uid

想定フィールド

- organizationId : 所属組織ID
- uid : 認証UID
- name : 氏名
- kana : フリガナ
- birthDate : 生年月日

- email : メールアドレス
- phone : 電話番号
- memberCode : 会員番号
- category : 会員区分
- status : 会員状態
- biometricEnabled : 生体認証有効フラグ
- lastLoginAt : 最終ログイン日時
- createdFromRegistrationId : 元申請ID
- createdAt : 作成日時
- updatedAt : 更新日時

status の初期想定

- active
- inactive
- suspended

サンプル

```
{
  "organizationId": "org001",
  "uid": "authUid123",
  "name": "山田太郎",
  "kana": "ヤマダタロウ",
  "birthDate": "1980-05-10",
  "email": "sample@example.com",
  "phone": "09012345678",
  "memberCode": "A-001",
  "category": "一般会員",
  "status": "active",
  "biometricEnabled": true,
  "lastLoginAt": "Timestamp",
  "createdFromRegistrationId": "reg123",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

運用ルール

- 会員ページ表示可否は members/{uid} の存在と status == active で判定する
- 未承認ユーザーはこのドキュメントを持たない
- 顔認証は端末側で行い、Firestore 側では biometricEnabled を補助状態として保持する

17-6. organizations/{organizationId}/messages/{messageId}

役割

管理者から会員へ送るメッセージ本文を保存する。

ドキュメントID

- 自動採番

想定フィールド

- `organizationId` : 所属組織ID
- `title` : 件名
- `body` : 本文
- `targetType` : 配信対象種別
- `isPublished` : 公開状態
- `publishedAt` : 公開日時
- `createdByUid` : 作成管理者UID
- `updatedByUid` : 更新管理者UID
- `createdAt` : 作成日時
- `updatedAt` : 更新日時

`targetType` の初期想定

- `allMembers`

※ 初期版では全会員一斉配信のみを採用する。個別配信やカテゴリ配信は将来拡張とする。

サンプル

```
{
  "organizationId": "org001",
  "title": "今月のお知らせ",
  "body": "今月のご案内です。",
  "targetType": "allMembers",
  "isPublished": true,
  "publishedAt": "Timestamp",
  "createdByUid": "adminUid001",
  "updatedByUid": "adminUid001",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

運用ルール

- 会員アプリは `isPublished == true` のみ読む
- 一覧は `publishedAt desc` で表示する
- 既読状態は message ドキュメント本体に持たせず別管理する

17-7. organizations/{organizationId}/members/{uid}/messageReads/{messageId}

役割

各会員ごとの既読状態を管理する。

ドキュメントID

- messageId

想定フィールド

- messageId : メッセージID
- readAt : 既読日時
- createdAt : 作成日時

サンプル

```
{
  "messageId": "msg001",
  "readAt": "Timestamp",
  "createdAt": "Timestamp"
}
```

この構成を採用する理由

- メッセージ本体に既読UID配列を持たせるより安全で拡張しやすい
- 会員本人だけが自分の既読情報を書ける
- 将来、既読率集計も管理側で計算しやすい

17-8. organizations/{organizationId}/videos/{videoId}

役割

販売対象となる動画コンテンツ情報を保存する。

ドキュメントID

- 自動採番

想定フィールド

- organizationId : 所属組織ID
- title : 動画タイトル
- description : 説明文
- thumbnailURL : サムネイル画像URL
- videoProvider : 動画配信サービス名

- videoSourceId : 外部動画ID
- videoURL : 再生用URLまたは埋め込みURL
- price : 価格
- currency : 通貨
- isPublished : 公開状態
- sortOrder : 表示順
- createdByUid : 作成管理者UID
- updatedByUid : 更新管理者UID
- createdAt : 作成日時
- updatedAt : 更新日時

サンプル

```
{
  "organizationId": "org001",
  "title": "基礎講座1",
  "description": "初めての方向け動画です。",
  "thumbnailURL": "https://...",
  "videoProvider": "vimeo",
  "videoSourceId": "123456789",
  "videoURL": "https://player.vimeo.com/video/123456789",
  "price": 3000,
  "currency": "JPY",
  "isPublished": true,
  "sortOrder": 10,
  "createdByUid": "adminUid001",
  "updatedByUid": "adminUid001",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

運用ルール

- 会員アプリは isPublished == true のみ表示する
- 動画URLの直保存は初期版では可とする
- 将来は Cloud Functions 経由の署名URLや視聴トークン方式へ拡張可能とする

17-9. organizations/{organizationId}/purchases/{purchaseId}

役割

会員の動画購入情報を管理する。

ドキュメントID

- 自動採番

想定フィールド

- `organizationId` : 所属組織ID
- `uid` : 購入会員UID
- `videoId` : 対象動画ID
- `paymentStatus` : 決済状態
- `price` : 購入価格
- `currency` : 通貨
- `purchasedAt` : 購入日時
- `expiresAt` : 有効期限
- `createdAt` : 作成日時
- `updatedAt` : 更新日時

`paymentStatus` の初期想定

- `paid`
- `pending`
- `cancelled`
- `refunded`

サンプル

```
{
  "organizationId": "org001",
  "uid": "authUid123",
  "videoId": "video001",
  "paymentStatus": "paid",
  "price": 3000,
  "currency": "JPY",
  "purchasedAt": "Timestamp",
  "expiresAt": null,
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

運用ルール

- 初期版では 1動画1購入 を前提とする
- 同一会員が同一動画を重複購入しないよう、アプリ側または Functions 側で制御する
- 視聴可否は `paymentStatus == paid` で判定する

17-10. organizations/{organizationId}/members/{uid}/videoAccess/{videoId}

役割

会員アプリが高速に視聴可否を判定するための補助データ。

必須ではないが推奨

購入履歴を毎回全件検索せずに済むため、初期版でも採用価値が高い。

想定フィールド

- `videoId` : 動画ID
- `hasAccess` : 視聴可否
- `grantedBy` : 付与元
- `grantedAt` : 付与日時
- `expiresAt` : 有効期限
- `updatedAt` : 更新日時

`grantedBy` の初期想定

- `purchase`
- `admin`

サンプル

```
{
  "videoId": "video001",
  "hasAccess": true,
  "grantedBy": "purchase",
  "grantedAt": "Timestamp",
  "expiresAt": null,
  "updatedAt": "Timestamp"
}
```

運用ルール

- 購入完了時に `videoAccess/{videoId}` を作成または更新する
- 会員アプリではこのドキュメントの有無で視聴ボタン表示を切り替えられる

17-11. organizations/{organizationId}/system/version

役割

アプリ動作や将来移行のための内部管理情報。

想定フィールド

- `schemaVersion` : スキーマバージョン
- `minimumAppVersion` : 最低利用アプリ版
- `updatedAt` : 更新日時

サンプル

```
{
  "schemaVersion": 1,
  "minimumAppVersion": "1.0.0",
  "updatedAt": "Timestamp"
}
```

18. ER的な関係整理

```
organizations
├── organizationId
│   ├── settings/public
│   ├── admins/{uid}
│   ├── memberRegistrations/{registrationId}
│   ├── members/{uid}
│   │   ├── messageReads/{messageId}
│   │   └── videoAccess/{videoId}
│   ├── messages/{messageId}
│   ├── videos/{videoId}
│   ├── purchases/{purchaseId}
│   └── system/version
```

主な関連

- `memberRegistrations.createdByUid` → 認証ユーザー
- `members.uid` → Firebase Auth UID
- `messageReads/{messageId}` → `messages/{messageId}` と対応
- `purchases.uid` → `members/{uid}` と対応
- `purchases.videoId` → `videos/{videoId}` と対応
- `videoAccess/{videoId}` → `videos/{videoId}` と対応

19. クライアントからの主な読み書き

19-1. 会員アプリが読むもの

- `organizations/{organizationId}`
- `organizations/{organizationId}/settings/public`
- `organizations/{organizationId}/members/{uid}`
- `organizations/{organizationId}/messages/*`
- `organizations/{organizationId}/members/{uid}/messageReads/*`
- `organizations/{organizationId}/videos/*`
- `organizations/{organizationId}/members/{uid}/videoAccess/*`

19-2. 会員アプリが書くもの

- `organizations/{organizationId}/memberRegistrations/{registrationId}` 作成
- `organizations/{organizationId}/members/{uid}/messageReads/{messageId}` 作成
- 必要に応じて `members/{uid}.lastLoginAt` 更新

19-3. 管理機能が読むもの

- `memberRegistrations`
- `members`
- `messages`
- `videos`
- `purchases`

19-4. 管理機能が書くもの

- `admins`
 - `memberRegistrations` 更新
 - `members` 作成・更新
 - `messages` 作成・更新
 - `videos` 作成・更新
 - `purchases` 作成・更新
 - `members/{uid}/videoAccess/{videoId}` 作成・更新
-

20. インデックス推奨

初期版で想定される主なクエリに対して、以下を推奨する。

20-1. memberRegistrations

- `status asc, createdAt desc`

20-2. members

- `status asc, name asc`
- `category asc, name asc` (将来必要なら)

20-3. messages

- `isPublished asc, publishedAt desc`

20-4. videos

- `isPublished asc, sortOrder asc`
- `isPublished asc, createdAt desc`

20-5. purchases

- `uid asc, purchasedAt desc`
- `videoId asc, purchasedAt desc`

21. ドキュメントIDの推奨ルール

UID固定推奨

以下は Auth UID をドキュメントIDに使う。

- `admins/{uid}`
- `members/{uid}`

自動採番推奨

以下は Firestore 自動IDを使う。

- `memberRegistrations/{registrationId}`
- `messages/{messageId}`
- `videos/{videoId}`
- `purchases/{purchaseId}`

固定ID推奨

以下は固定IDでよい。

- `settings/public`
- `system/version`
- `members/{uid}/messageReads/{messageId}`
- `members/{uid}/videoAccess/{videoId}`

22. 初期版での認証と会員判定ロジック

22-1. 認証

- Firebase Authentication を利用する
- 初期版ではメール認証または匿名認証+会員申請紐付けのどちらでも構成可能
- 管理のしやすさを考えると、将来的にはメールアドレス基準の認証へ寄せる方が望ましい

22-2. 会員判定

会員アプリでは以下の条件を満たした場合のみ会員ページを開ける。

1. 認証済みである
2. `members/{uid}` が存在する
3. `status == active` である
4. 端末側の生体認証に成功する

22-3. 未承認ユーザー

- `memberRegistrations` は存在しても `members/{uid}` が無ければ会員ページへ入れない

23. 初期版での購入判定ロジック

動画の購入済み判定は、次の順で見る。

1. `members/{uid}/videoAccess/{videoId}` を確認
2. `hasAccess == true` なら視聴可
3. 無ければ未購入扱い

補足

- `purchases` は履歴管理用
- `videoAccess` は表示と視聴判定用
- この二層構造にすると、会員アプリ側が単純になる

24. 初期版でのおすすめ実装範囲

初期版では以下の構成を採用すると、最も安定して作りやすい。

必須採用コレクション

- `organizations`
- `settings/public`
- `admins`
- `memberRegistrations`
- `members`
- `messages`
- `videos`
- `purchases`
- `members/{uid}/messageReads`
- `members/{uid}/videoAccess`

初期版では作らないもの

- `categories`
- `branches`
- `announcements`
- `diaryEntries`
- `events`
- `advancedPermissions`
- `subscriptions`

25. 実装にそのまま使える最終構成

```
organizations/{organizationId}
  name
  description
  logoURL
```

openingImageUrl
supportEmail
isActive
createdAt
updatedAt

organizations/{organizationId}/settings/public
displayName
openingEnabled
openingImageUrl
themeColorHex
registrationEnabled
videoSalesEnabled
messageEnabled
updatedAt

organizations/{organizationId}/admins/{uid}
name
email
role
isActive
createdAt
updatedAt

organizations/{organizationId}/memberRegistrations/{registrationId}
organizationId
name
kana
birthDate
email
phone
memberCode
category
note
status
createdByUid
reviewedByUid
reviewedAt
createdAt
updatedAt

organizations/{organizationId}/members/{uid}
organizationId
uid
name
kana
birthDate
email
phone
memberCode
category

status
biometricEnabled
lastLoginAt
createdFromRegistrationId
createdAt
updatedAt

organizations/{organizationId}/members/{uid}/messageReads/{messageId}
messageId
readAt
createdAt

organizations/{organizationId}/members/{uid}/videoAccess/{videoId}
videoId
hasAccess
grantedBy
grantedAt
expiresAt
updatedAt

organizations/{organizationId}/messages/{messageId}
organizationId
title
body
targetType
isPublished
publishedAt
createdByUid
updatedByUid
createdAt
updatedAt

organizations/{organizationId}/videos/{videoId}
organizationId
title
description
thumbnailURL
videoProvider
videoSourceId
videoURL
price
currency
isPublished
sortOrder
createdByUid
updatedByUid
createdAt
updatedAt

organizations/{organizationId}/purchases/{purchaseId}
organizationId

```
uid
videoId
paymentStatus
price
currency
purchasedAt
expiresAt
createdAt
updatedAt
```

```
organizations/{organizationId}/system/version
  schemaVersion
  minimumAppVersion
  updatedAt
```

26. 設計結論

この Firestore 設計図は、今回の新アプリの目的である

- 会員登録
- 顔認証ログイン
- メッセージ
- ビデオコンテンツ販売

の4機能を、**無理なく、壊れにくく、将来拡張しやすい形**で実装するための完成版である。

特に重要なポイントは以下である。

- すべてを `organizations/{organizationId}` 配下へ統一する
- 承認前は `memberRegistrations`、承認後は `members` に分離する
- 既読は `members/{uid}/messageReads` に持つ
- 購入履歴は `purchases`、視聴可否は `videoAccess` に分ける
- 初期版では機能を増やしすぎない

この構成を基準にすれば、次に **Firestore Rules 最終版**、**Storage Rules 最終版**、**Swiftファイル骨組み一覧** へそのまま進められる。